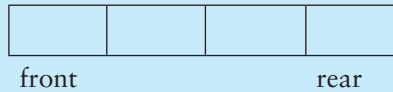


25. Suppose the following operations are performed on an empty queue:

```
enqueue(5);
enqueue(7);
dequeue();
enqueue(9);
enqueue(12);
dequeue();
enqueue(10);
```

Insert numbers in the following diagram to show what will be stored in the static stack after the operations above have executed.



26. What problem is overcome by using a circular array for a static queue?
27. Write two different code segments that may be used to wrap an index back around to the beginning of an array when it moves past the end of the array. Use an `if/else` statement in one segment and modular arithmetic in the other.

### True or False

28. T    F    A static stack or queue is built around an array.
29. T    F    The size of a dynamic stack or queue must be known in advance.
30. T    F    The push operation inserts an element at the end of a stack.
31. T    F    The pop operation retrieves an element from the top of a stack.
32. T    F    The STL stack container's pop operation does not retrieve the top element of the stack, it just removes it.

## Programming Challenges

### 1. Static Stack Template

Write your own version of a class template that will create a static stack of any data type. Demonstrate the class with a driver program.

### 2. Dynamic Stack Template

Write your own version of a class template that will create a dynamic stack of any data type. Demonstrate the class with a driver program.

3. Static Queue Template

Write your own version of a class template that will create a static queue of any data type. Demonstrate the class with a driver program.

4. Dynamic Queue Template

Write your own version of a class template that will create a dynamic queue of any data type. Demonstrate the class with a driver program.

5. Error Testing

The `DynIntStack` and `DynIntQueue` classes shown in this chapter are abstract data types using a dynamic stack and dynamic queue, respectively. The classes do not currently test for memory allocation errors. Modify the classes so they determine whether new nodes cannot be created by handling the `bad_alloc` exception.



**NOTE:** If you have already done Programming Challenges 2 and 4, modify the templates you created.

6. Dynamic String Stack

Design a class that stores strings on a dynamic stack. The strings should not be fixed in length. Demonstrate the class with a driver program.

7. Dynamic MathStack

The `MathStack` class shown in this chapter only has two member functions: `add` and `sub`. Write the following additional member functions:

| Function             | Description                                                                                                           |
|----------------------|-----------------------------------------------------------------------------------------------------------------------|
| <code>mult</code>    | Pops the top two values off the stack, multiplies them, and pushes their product onto the stack.                      |
| <code>div</code>     | Pops the top two values off the stack, divides the second value by the first, and pushes the quotient onto the stack. |
| <code>addAll</code>  | Pops all values off the stack, adds them, and pushes their sum onto the stack.                                        |
| <code>multAll</code> | Pops all values off the stack, multiplies them, and pushes their product onto the stack.                              |

Demonstrate the class with a driver program.

8. Dynamic MathStack Template

Currently the `MathStack` class is derived from the `IntStack` class. Modify it so it is a template, derived from the template you created in Programming Challenge 2.

9. File Reverser

Write a program that opens a text file and reads its contents into a stack of characters. The program should then pop the characters from the stack and save them in a second text file. The order of the characters saved in the second file should be the reverse of their order in the first file.

## 10. File Filter

Write a program that opens a text file and reads its contents into a queue of characters. The program should then dequeue each character, convert it to uppercase, and store it in a second file.

## 11. File Compare



Write a program that opens two text files and reads their contents into two separate queues. The program should then determine whether the files are identical by comparing the characters in the queues. When two nonidentical characters are encountered, the program should display a message indicating that the files are not the same. If both queues contain the same set of characters, a message should be displayed indicating that the files are identical.

## 12. Inventory Bin Stack

Design an inventory class that stores the following members:

|                            |                                                         |
|----------------------------|---------------------------------------------------------|
| <code>serialNum:</code>    | An integer that holds a part's serial number.           |
| <code>manufactDate:</code> | A member that holds the date the part was manufactured. |
| <code>lotNum:</code>       | An integer that holds the part's lot number.            |

The class should have appropriate member functions for storing data into, and retrieving data from, these members.

Next, design a stack class that can hold objects of the class described above. If you wish, you may use the template you designed in Programming Challenge 1 or 2.

Last, design a program that uses the stack class described above. The program should have a loop that asks the user if he or she wishes to add a part to inventory, or take a part from inventory. The loop should repeat until the user is finished.

If the user wishes to add a part to inventory, the program should ask for the serial number, date of manufacture, and lot number. The data should be stored in an inventory object, and pushed onto the stack.

If the user wishes to take a part from inventory, the program should pop the top-most part from the stack and display the contents of its member variables.

When the user finishes the program, it should display the contents of the member values of all the objects that remain on the stack.

## 13. Inventory Bin Queue

Modify the program you wrote for Programming Challenge 12 so it uses a queue instead of a stack. Compare the order in which the parts are removed from the bin for each program.

## 14. Balanced Parentheses

A string of characters has balanced parentheses if each right parenthesis occurring in the string is matched with a preceding left parenthesis, in the same way that each right brace in a C++ program is matched with a preceding left brace. Write a program that uses a stack to determine whether a string entered at the keyboard has balanced parentheses.

**15. Balanced Multiple Delimiters**

A string may use more than one type of delimiter to bracket information into "blocks." For example, A string may use braces { }, parentheses ( ), and brackets [ ] as delimiters. A string is properly delimited if each right delimiter is matched with a preceding left delimiter of the same type in such a way that either the resulting blocks of information are disjoint, or one of them is completely nested within the other. Write a program that uses a single stack to check whether a string containing braces, parentheses, and brackets is properly delimited.